BOOKLEAF PUBLISHING — AUTHOR SUPPORT PORTAL

Architectural Approach & Technical Write-Up

Date: October 2026 | **Stack:** Next.js 16, Mongo, Gemini AI

Candidate: Vaibhav Agarwal
Role: Full-Stack Developer

1. APPROACH & CORE PRIORITIES

When building the BookLeaf Author Support Portal, I prioritized **operational reliability, deterministic data consistency, and practical author trust** over superficial cosmetic complexity. A customer support platform succeeds or fails on response latency, data clarity, and zero message drop.

• **Unbreakable Ticket Pipeline:** Author tickets must never be blocked or lost. If Google Gemini experiences rate limits (HTTP 429) or transient network downtime, the system implements exponential backoff with jitter and gracefully falls back to a human-triaged pending state while saving the author's message intact.

• **Policy-Grounded, Calibrated AI:** Rather than allowing free-form hallucination, Gemini 2.5 Flash is strictly prompt-engineered to embody BookLeaf's author-first collaborative tone and bound by verified business rules: 80/20 royalty splits, 45-day quarterly settlement cycles, 48-hour ISBN escalations, and free defect reprints.

• **Financial Transparency & Real Data:** Authors can track copies sold, earned royalties, and pending payouts with live mathematical consistency (Royalty/Copy × Copies Sold = Total Earned). The admin dashboard replaced mock figures with live MongoDB aggregation and instant Excel export.

2. KEY ARCHITECTURAL TRADE-OFFS

• **Dedicated Socket.IO Server with 6s Polling Fallback:** Next.js App Router API routes are stateless and cannot sustain persistent WebSockets in serverless environments (Vercel). I implemented a companion Node WebSocket server (port 4001) for local zero-latency push, paired with an automatic 6-second client polling fallback. This guarantees real-time notification bells and live conversation streaming on serverless Vercel deployments with zero configuration.

• **Deterministic Policy Slicing vs Monolithic Vector DB:** Setting up an external vector database (Pinecone/Milvus) for a 6KB knowledge base introduces unnecessary network latency and operational cost. Instead, I built a deterministic category-routed policy module that injects only relevant rules into prompts. This achieves ~70% token cost savings ($0.075/1M tokens) with zero vector latency.

• **Friction-Free Mock Attachment Interface:** In strict accordance with the assessment brief, file upload UI is implemented cleanly on the client without forcing third-party cloud S3 credentials during local reviewer evaluation.

3. PRODUCTION EVOLUTION ROADMAP (SCALE TO 22,000+ BOOKS)

• **Cloud Object Storage (AWS S3 / Cloudflare R2):** Integrate presigned upload URLs with automated ClamAV antivirus scanning for physical defect photo proofs and high-resolution manuscript attachments.

• **Redis Message Broker for WebSocket Clustering:** Connect @socket.io/redis-adapter backed by Redis/Upstash to distribute real-time events across horizontally-scaled Docker/Kubernetes container pods.

• **Asynchronous Job Queue (BullMQ / Inngest):** Decouple ticket ingestion from AI processing by placing classification and draft generation onto background workers with Dead Letter Queues (DLQ) to protect API concurrency during viral traffic spikes.

• **Automated Distribution Webhooks:** Ingest automated inventory and listing webhooks directly from Amazon KDP, IngramSpark, and Flipkart to proactively resolve stock and listing inquiries without manual author tickets.